SortedMap interface

24 May 2025 20:50

Map (I)
↑
SortedMap (I).

It is the child interface of Map interface. If we want to represent a group of key-value pair according to some sorting order of keys, then we should go for **SortedMap** interface. Sorting is based on key, but not based on value.

Object firstKey() → 101
Object lastKey() → 116
SortedMap headMap(104) → {101=A, 102=B, 103=C}.
SortedMap tailMap(104) → {104=D, 105=E, 116=F}.
SortedMap subMap(103, 116) ⇒ {103=C, 104=D, 105=E}
Comparator comparator() → null.

Key	Value
101	A
102	B
103	C
104	D
105	E
116	F

Official documentations:
public interface SortedMap<K,V>
extends SequencedMap<K,V>
A Map that further provides a total ordering on its keys. The map is ordered according to the natural ordering of its keys, or by a Comparator typically provided at sorted map creation time. This order is reflected when iterating over the sorted map's collection views (returned by the entrySet, keySet and values methods). Several additional operations are provided to take advantage of the ordering. (This interface is the map analogue of SortedSet.)

All keys inserted into a sorted map must implement the Comparable interface (or be accepted by the specified comparator). Furthermore, all such keys must be mutually comparable: k1.compareTo(k2) (or comparator.compare(k1, k2)) must not throw a ClassCastException for any keys k1 and k2 in the sorted map. Attempts to violate this restriction will cause the offending method or constructor invocation to throw a ClassCastException.

Note that the ordering maintained by a sorted map (whether or not an explicit comparator is provided) must be consistent with equals if the sorted map is to correctly implement the Map interface. (See the Comparable interface or Comparator interface for a precise definition of consistent with equals.) This is so because the Map interface is defined in terms of the equals operation, but a sorted map performs all key comparisons using its compareTo (or compare) method, so two keys that are deemed equal by this method are, from the standpoint of the sorted map, equal. The behavior of a tree map is well-defined even if its ordering is inconsistent with equals; it just fails to obey the general contract of the Map interface.

All general-purpose sorted map implementation classes should provide four "standard" constructors. It is not possible to enforce this recommendation though as required constructors cannot be specified by interfaces. The expected "standard" constructors for all sorted map implementations are:

A void (no arguments) constructor, which creates an empty sorted map sorted according to the natural ordering of its keys.
A constructor with a single argument of type Comparator, which creates an empty sorted map sorted according to the specified comparator.
A constructor with a single argument of type Map, which creates a new map with the same key-value mappings as its

argument, sorted according to the keys' natural ordering.
A constructor with a single argument of type SortedMap, which creates a new sorted map with the same key-value mappings and the same ordering as the input sorted map.

Note: several methods return submaps with restricted key ranges. Such ranges are half-open, that is, they include their low endpoint but not their high endpoint (where applicable). If you need a closed range (which includes both endpoints), and the key type allows for calculation of the successor of a given key, merely request the subrange from lowEndpoint to successor(highEndpoint). For example, suppose that m is a map whose keys are strings. The following idiom obtains a view containing all of the key-value mappings in m whose keys are between low and high, inclusive:

```
SortedMap<String, V> sub = m.subMap(low, high+"\0");
```

A similar technique can be used to generate an open range (which contains neither endpoint). The following idiom obtains a view containing all of the key-value mappings in m whose keys are between low and high, exclusive:

```
SortedMap<String, V> sub = m.subMap(low+"\0", high);
```

This interface is a member of the Java Collections Framework.

Since:
1.2

All Methods | Instance Methods | Abstract Methods | Default Methods

Modifier and Type	Method	Description
Comparator<? super K>	comparator()	Returns the comparator used to order the keys in this map, or null if this map uses the natural ordering of its keys.
Set<Map.Entry<K,V>>	entrySet()	Returns a Set view of the mappings contained in this map.
K	firstKey()	Returns the first (lowest) key currently in this map.
SortedMap<K,V>	headMap(K toKey)	Returns a view of the portion of this map whose keys are strictly less than toKey.
Set<K>	keySet()	Returns a Set view of the keys contained in this map.
K	lastKey()	Returns the last (highest) key currently in this map.
default V	putFirst(K k, V v)	Throws UnsupportedOperationException.
default V	putLast(K k, V v)	Throws UnsupportedOperationException.
default SortedMap<K,V>	reversed()	Returns a reverse-ordered view of this map.
SortedMap<K,V>	subMap(K fromKey, K toKey)	Returns a view of the portion of this map whose keys range from fromKey, inclusive, to toKey, exclusive.
SortedMap<K,V>	tailMap(K fromKey)	Returns a view of the portion of this map whose keys are greater than or equal to fromKey.
Collection<V>	values()	Returns a Collection view of the values contained in this map.

TreeMap

The underlying data structure is red-black tree.
Insertion order is not preserved and it is based on some sorting order of keys.
Duplicate keys are not allowed, but values can be duplicated.
If we are depending on default natural sorting order then. keys should be homogeneous and comparable. Otherwise we will get runtime exceptions, saying ClassCastException.
If we are defining our own sorting by Comparator, then keys need not be homogeneous and Comparable. We can take heterogeneous, non-Comparable objects also.
Whether we are depending on default natural sorting order or customized sorting order, there are no restriction for values. We can store heterogeneous non Comparable objects also.

Null is not accepted at all in keys.

Constructor of TreeMap

Treemap t = new TreeMap(); => Here default natural sorting of key is used.
Treemap t = new TreeMap(Comparator c); =>Here we use customized sorting order
Treemap t = new TreeMap(Map m);
Treemap t = new TreeMap(SortedMap m);

Code:

```
package Chapter5;
import java.util.TreeMap;
public class TreeMapDemo {
    public static void main(String[] args) {
        TreeMap m = new TreeMap<>();
        m.put(100, "Aman");
        m.put(87, "Naman");
        m.put(112, "Aditya");
        m.put(95,"Anchit");
        m.put(98, "Sanjay");
        m.put(20,"Subhalakshmi");
        System.out.println(m);
    }
}
```

Output:
{20=Subhalakshmi, 87=Naman, 95=Anchit, 98=Sanjay, 100=Aman, 112=Aditya}

Inserting 'null value' in key and value we get NullPointerException

```
package Chapter5;

import java.util.TreeMap;

public class TreeMapDemo {
   public static void main(String[] args) {
      TreeMap m = new TreeMap<>();
      m.put(100, "Aman");
      m.put(87, "Naman");
      m.put(112, "Aditya");
      m.put(95,"Anchit");
      m.put(98, "Sanjay");
      m.put(20,"Subhalakshmi");
      m.put(null, null);  ---> NullPointerException.
      System.out.println(m);
   }
}
```

Output:

```
Exception in thread "main" java.lang.NullPointerException
        at java.base/java.util.Objects.requireNonNull(Objects.java:233)
        at java.base/java.util.TreeMap.put(TreeMap.java:844)
        at java.base/java.util.TreeMap.put(TreeMap.java:569)
        at Chapter5.TreeMapDemo.main(TreeMapDemo.java:14)
```

```java
package Chapter5;
import java.util.TreeMap;
public class TreeMapDemo {
    public static void main(String[] args) {
        TreeMap m = new TreeMap<>();
        m.put(100, "Aman");
        m.put(87, "Naman");
        m.put(112, "Aditya");
        m.put(95,"Anchit");
        m.put(98, "Sanjay");
        m.put(20,"Subhalakshmi");
        m.put(170, null);
        m.put(130, null);
        System.out.println(m);
    }
}
```

Output:
{20=Subhalakshmi, 87=Naman, 95=Anchit, 98=Sanjay, 100=Aman, 112=Aditya, 130=null, 170=null}

ClassCastException is thrown by JVM when we try to store heterogenous objects in TreeMap object.

```java
package Chapter5;
import java.util.TreeMap;
public class TreeMapDemo {
    public static void main(String[] args) {
        TreeMap m = new TreeMap<>();
        m.put(100, "Aman");
        m.put(87, "Naman");
        m.put(112, "Aditya");
        m.put(95,"Anchit");
        m.put(98, "Sanjay");
        m.put(20,"Subhalakshmi");
        m.put(170, null);
        m.put(130, null);
        System.out.println(m);
        m.put("Varun", "Arun");
    }
}
```

Heterogenous Object in key causes ClassCastException (runtime)

Output:
{20=Subhalakshmi, 87=Naman, 95=Anchit, 98=Sanjay, 100=Aman, 112=Aditya, 130=null, 170=null}
Exception in thread "main" java.lang.ClassCastException: class java.lang.Integer cannot be cast to class java.lang.String (java.lang.Integer and java.lang.String are in module java.base of loader 'bootstrap')
    at java.base/java.lang.String.compareTo(String.java:142)
    at java.base/java.util.TreeMap.put(TreeMap.java:849)
    at java.base/java.util.TreeMap.put(TreeMap.java:569)
    at Chapter5.TreeMapDemo.main(TreeMapDemo.java:17)

```java
package Chapter5;
import java.util.Comparator;

public class MyComparator implements Comparator {
  @Override
  public int compare(Object o1, Object o2){
    return o2.toString().compareTo(o1.toString());
  }
}
```

```
package Chapter5;
import java.util.TreeMap;
public class TreeMapDemoForComparator {
    public static void main(String[] args) {
        TreeMap t = new TreeMap(new MyComparator());
        t.put(100, "Ajit");
        t.put("Sachin", "Tendulkar");
        t.put(99.96, "Don Bradman");
        t.put(234678855467L, "big number");
        System.out.println(t);
    }
}
```

We have stored Hetrogenous object in TreeMap object, because we have passed Comparator object in the constructor of TreeMap, while creating the TreeMap object.

```
Output:
    {Sachin=Tendulkar, 99.96=Don Bradman, 234678855467=big number, 100=Ajit}
```

Hashtable

The underlying data structure for **Hashtable** is hash table.
Insertion order is not preserved, and it is based on HashCode of keys.
Duplicate keys are not allowed and values can be duplicated.
Heterogeneous objects are allowed for both keys and values.
Null is not allowed for both key and value, otherwise we will get runtime exception saying **NullPointerException**.
It implements **Serializable, Map<k><v>** and **Cloneable** interfaces, but not **RandomAccess**.
Every method present in hash table is **synchronized** and hence, HashTable object is Thread safe.
HashTable is the best choice if our frequent operation is retrieval or search operation.

Constructors

Hashtable h = new Hashtable(); ==> Creates an empty Hashtable object with default initial capacity 11 and default fill ratio is 0.75.
Hashtable h = new Hashtable(int initialCapacity, float fillRatio); ==> Creates an empty Hashtable object with given initial capacity and fill ratio. The value of [0 <fillratio <= 1.0]

Hashtable h = new Hashtable(int initialCapacity); ==> Creates an empty Hashtable object with given initial capacity and default fill ratio is 0.75.

Hashtable h = new Hashtable(Map m); ==> this constructor interconvert any Map object to Hashtable object.

public class Hashtable<K,V> extends Dictionary<K,V> implements Map<K,V>, Cloneable, Serializable
This class implements a hash table, which maps keys to values. Any non-null object can be used as a key or as a value.
To successfully store and retrieve objects from a hashtable, the objects used as keys must implement the hashCode method and the equals method.
An instance of Hashtable has two parameters that affect its performance: *initial capacity* and *load factor*. The *capacity* is the number of *buckets* in the hash table, and the *initial capacity* is simply the capacity at the time the hash table is created. Note that the hash table is *open*: in the case of a "hash collision", a single bucket stores multiple entries, which must be searched sequentially. The *load factor* is a measure of how full the hash table is allowed to get before its capacity is automatically increased. The initial capacity and load factor parameters are merely hints to the implementation. The exact details as to when and whether the rehash method is invoked are implementation-dependent.
Generally, the default load factor (.75) offers a good tradeoff between time and space costs. Higher values decrease the space overhead but increase the time cost to look up an entry (which is reflected in most Hashtable operations, including get and put).
The initial capacity controls a tradeoff between wasted space and the need for rehash operations, which are time-consuming. No rehash operations will *ever* occur if the initial capacity is greater than the maximum number of entries the

Hashtable will contain divided by its load factor. However, setting the initial capacity too high can waste space.
If many entries are to be made into a Hashtable, creating it with a sufficiently large capacity may allow the entries to be inserted more efficiently than letting it perform automatic rehashing as needed to grow the table.
This example creates a hashtable of numbers. It uses the names of the numbers as keys:

```
Hashtable<String, Integer> numbers
  = new Hashtable<String, Integer>();
numbers.put("one", 1);
numbers.put("two", 2);
numbers.put("three", 3);
```

To retrieve a number, use the following code:

```
Integer n = numbers.get("two");
if (n != null) {
  System.out.println("two = " + n);
}
```

The iterators returned by the iterator method of the collections returned by all of this class's "collection view methods" are *fail-fast*: if the Hashtable is structurally modified at any time after the iterator is created, in any way except through the iterator's own remove method, the iterator will throw a ConcurrentModificationException. Thus, in the face of concurrent modification, the iterator fails quickly and cleanly, rather than risking arbitrary, non-deterministic behavior at an undetermined time in the future. The Enumerations returned by Hashtable's keys and elements methods are *not* fail-fast; if the Hashtable is structurally modified at any time after the enumeration is created then the results of enumerating are undefined.
Note that the fail-fast behavior of an iterator cannot be guaranteed as it is, generally speaking, impossible to make any hard guarantees in the presence of unsynchronized concurrent modification. Fail-fast iterators throw ConcurrentModificationException on a best-effort basis. Therefore, it would be wrong to write a program that depended on this exception for its correctness: *the fail-fast behavior of iterators should be used only to detect bugs.*
As of the Java 2 platform v1.2, this class was retrofitted to implement the Map interface, making it a member of the Java Collections Framework. Unlike the new collection implementations, Hashtable is synchronized. If a thread-safe implementation is not needed, it is recommended to use HashMap in place of Hashtable. If a thread-safe highly-concurrent implementation is desired, then it is recommended to use ConcurrentHashMap in place of Hashtable.

Since:

1.0

From <https://docs.oracle.com/en/java/javase/24/docs/api//java.base/java/util/Hashtable.html>

Click the link and check out the methods present in HashTable class implemented all the methods of Map class.

Demo Code:

```
package Chapter5;
public class Key {
    int i;
    public Key(int i) {
        this.i = i;
    }
    @Override
    public int hashCode() {
        return i;
    }
    @Override
    public boolean equals(Object obj) {
        Key k = null;
        if(obj instanceof Key){
            k = (Key)obj;
        }
        System.out.println("In equals");
        if(this.i == k.i){
```

→ Any hashCode algorithm can be implemented here..

Overriding of these methods are compulsory to smooth running of Hashtable executions.

```
        }
        System.out.println("In equals");
        if(this.i == k.i){
            return true;
        }
        else{
            return false;
        }
    }
    @Override
    public String toString() {
        return "Key [i=" + i + "]";
    }

}
```

are compulsory to smooth running of Hashtable executions.

```
package Chapter5;
import java.util.Hashtable;
public class HashtableDemo {
    public static void main(String[] args) {
        Hashtable<Key, String> t = new Hashtable<Key, String>(25);
        t.put(new Key(5), "A");
        t.put(new Key(2), "B");
        t.put(new Key(25), "C");
        t.put(new Key(52), "D");
        t.put(new Key(16), "E");
        t.put(new Key(15), "F");
        t.put(new Key(12), "G");
        System.out.println(t);
        Key k1 = new Key(15);
        System.out.println("\n"+t.get(k1));

    }
}
```

→ how we can store <Key>, <value> pair in Hashtable object.

↑ retrive the value from Hashtable object

Output:
{Key [i=16]=E, Key [i=15]=F, Key [i=12]=G, Key [i=5]=A, Key [i=52]=D, Key [i=2]=B, Key [i=25]=C}
In equals

F